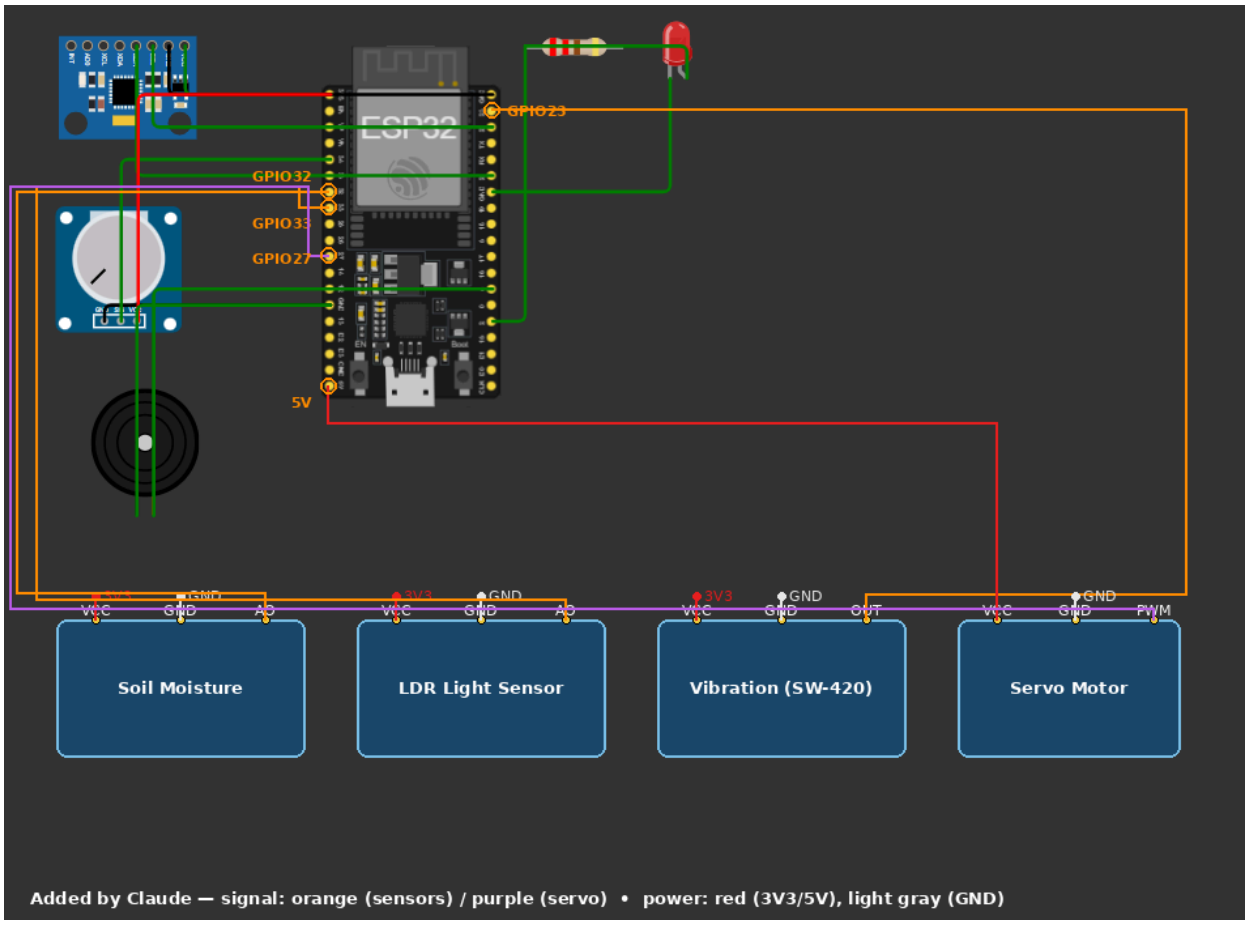


CODE FOR NODE 1 :

```
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <ESP32Servo.h>
#include <math.h>

Adafruit_MPU6050 mpu;
Servo warningServo;

// ======================================================
// NODE ID
// ======================================================

const int NODE_ID = 1;


// ======================================================
// WIFI
```

```
// ====================================================

const char* ssid = "Landslide_Central";
const char* password = "12345678";

const char* centralIP = "192.168.4.1";

WiFiClient client;


// ====================================================
// PINS
// ====================================================

const int potPin = 34;
const int vibrationPin = 35;
const int ldrPin = 32;
const int soilPin = 33;

const int ledPin = 2;
const int buzzerPin = 4;

const int servoPin = 27;


// ====================================================
// THRESHOLDS
// ====================================================

// Potentiometer
const int POT_THRESHOLD = 2500;

// Soil moisture
const int SOIL_THRESHOLD = 2500;

// MPU6050 movement threshold
const float ACCELERATION_THRESHOLD = 12.0;


// ====================================================
// HOW MANY SIGNALS ARE REQUIRED?
// ====================================================

const int REQUIRED_SIGNALS = 2;
```

```
// ======================================================
// SERVO POSITIONS
// ======================================================

const int SERVO_NORMAL = 0;
const int SERVO_WARNING = 90;


// ======================================================
// SETUP
// ======================================================

void setup() {

  Serial.begin(115200);


  // ----------------------------------------------------
  // SENSOR PINS
  // ----------------------------------------------------

  pinMode(vibrationPin, INPUT);


  // ----------------------------------------------------
  // LED + BUZZER
  // ----------------------------------------------------

  pinMode(ledPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);

  digitalWrite(ledPin, LOW);
  digitalWrite(buzzerPin, LOW);


  // ----------------------------------------------------
  // SERVO
  // ----------------------------------------------------

  warningServo.setPeriodHertz(50);

  warningServo.attach(
```

```
  servoPin,
  500,
  2400
);

warningServo.write(
  SERVO_NORMAL
);


// ------------------------------------------------
// I2C
// ------------------------------------------------

Wire.begin();


// ------------------------------------------------
// MPU6050
// ------------------------------------------------

if (!mpu.begin()) {

  Serial.println(
    "MPU6050 NOT FOUND!"
  );

  while (1) {
    delay(10);
  }
}

Serial.println(
  "MPU6050 connected!"
);


// ------------------------------------------------
// WIFI
// ------------------------------------------------

Serial.println();
Serial.println(
  "Connecting to Central Wi-Fi..."
```

```
);

WiFi.begin(
  ssid,
  password
);


while (
  WiFi.status() != WL_CONNECTED
) {

  delay(500);

  Serial.print(".");
}


Serial.println();

Serial.println(
  "Wi-Fi connected!"
);


Serial.print(
  "Node IP: "
);

Serial.println(
  WiFi.localIP()
);


Serial.print(
  "Central IP: "
);

Serial.println(
  centralIP
);


Serial.println();
```

```
  Serial.println(
    "================================="
  );

  Serial.println(
    "       NODE 1 ONLINE"
  );

  Serial.println(
    "   MULTI-SENSOR DETECTION"
  );

  Serial.println(
    "================================="
  );
}


// ======================================================
// LOOP
// ======================================================

void loop() {


  // ======================================================
  // READ SENSORS
  // ======================================================

  int potValue =
    analogRead(potPin);


  int vibrationValue =
    digitalRead(vibrationPin);


  int lightValue =
    analogRead(ldrPin);


  int soilValue =
    analogRead(soilPin);
```

```
// ================================================
// READ MPU6050
// ================================================

sensors_event_t a;
sensors_event_t g;
sensors_event_t temp;

mpu.getEvent(
  &a,
  &g,
  &temp
);


// ================================================
// CALCULATE TOTAL ACCELERATION
// ================================================

float acceleration =
  sqrt(
    a.acceleration.x *
    a.acceleration.x +

    a.acceleration.y *
    a.acceleration.y +

    a.acceleration.z *
    a.acceleration.z
  );


// ================================================
// INDIVIDUAL DETECTION SIGNALS
// ================================================

bool groundMovement = false;

bool vibrationDetected = false;

bool wetSoil = false;

bool mpuMovement = false;
```

```
// ------------------------------------------------
// POTENTIOMETER
// ------------------------------------------------

if (
  potValue >
  POT_THRESHOLD
) {

  groundMovement = true;
}


// ------------------------------------------------
// VIBRATION
// ------------------------------------------------

if (
  vibrationValue == HIGH
) {

  vibrationDetected = true;
}


// ------------------------------------------------
// SOIL
// ------------------------------------------------

if (
  soilValue >
  SOIL_THRESHOLD
) {

  wetSoil = true;
}


// ------------------------------------------------
// MPU6050
// ------------------------------------------------
```

```
if (
  acceleration >
  ACCELERATION_THRESHOLD
) {

  mpuMovement = true;
}


// ====================================================
// COUNT DETECTION SIGNALS
// ====================================================

int signalCount = 0;


if (groundMovement) {
  signalCount++;
}


if (vibrationDetected) {
  signalCount++;
}


if (wetSoil) {
  signalCount++;
}


if (mpuMovement) {
  signalCount++;
}


// ====================================================
// FINAL NODE DECISION
// ====================================================

bool abnormal =
  signalCount >= REQUIRED_SIGNALS;
```

```
// ================================================
// LOCAL LED / BUZZER / SERVO
// ================================================

if (abnormal) {

  digitalWrite(
    ledPin,
    HIGH
  );

  digitalWrite(
    buzzerPin,
    HIGH
  );

  warningServo.write(
    SERVO_WARNING
  );

}

else {

  digitalWrite(
    ledPin,
    LOW
  );

  digitalWrite(
    buzzerPin,
    LOW
  );

  warningServo.write(
    SERVO_NORMAL
  );
}


// ================================================
// SERIAL MONITOR
// ================================================
```

```
Serial.println();

Serial.println(
  "========== NODE 1 =========="
);


Serial.print(
  "Potentiometer : "
);

Serial.println(
  potValue
);


Serial.print(
  "Vibration     : "
);

Serial.println(
  vibrationValue
);


Serial.print(
  "Light         : "
);

Serial.println(
  lightValue
);


Serial.print(
  "Soil moisture : "
);

Serial.println(
  soilValue
);


Serial.print(
```

```
  "Acceleration  : "
);

Serial.print(
  acceleration
);

Serial.println(
  " m/s^2"
);


Serial.println(
  "---------------------------"
);


Serial.print(
  "Ground movement : "
);

Serial.println(
  groundMovement
    ? "YES"
    : "NO"
);


Serial.print(
  "Vibration       : "
);

Serial.println(
  vibrationDetected
    ? "YES"
    : "NO"
);


Serial.print(
  "Wet soil        : "
);

Serial.println(
```

```
  wetSoil
    ? "YES"
    : "NO"
);


Serial.print(
  "MPU movement    : "
);

Serial.println(
  mpuMovement
    ? "YES"
    : "NO"
);


Serial.print(
  "Detection signals = "
);

Serial.println(
  signalCount
);


Serial.print(
  "NODE STATUS = "
);


if (abnormal) {

  Serial.println(
    "ABNORMAL"
  );

}

else {

  Serial.println(
    "NORMAL"
  );
```

```
}


// ================================================
// SEND DATA TO CENTRAL
// ================================================

if (
  WiFi.status() ==
  WL_CONNECTED
) {


  if (
    client.connect(
      centralIP,
      80
    )
  ) {


    client.print(
      "NODE="
    );

    client.print(
      NODE_ID
    );


    client.print(
      ",POT="
    );

    client.print(
      potValue
    );


    client.print(
      ",VIB="
    );

    client.print(
```

```
  vibrationValue
);


client.print(
  ",LIGHT="
);

client.print(
  lightValue
);


client.print(
  ",SOIL="
);

client.print(
  soilValue
);


client.print(
  ",ACC="
);

client.print(
  acceleration
);


client.print(
  ",SIGNALS="
);

client.print(
  signalCount
);


client.print(
  ",ABNORMAL="
);
```

```
      client.println(
        abnormal
          ? 1
          : 0
      );


      client.stop();


      Serial.println(
        "Data sent to CENTRAL!"
      );

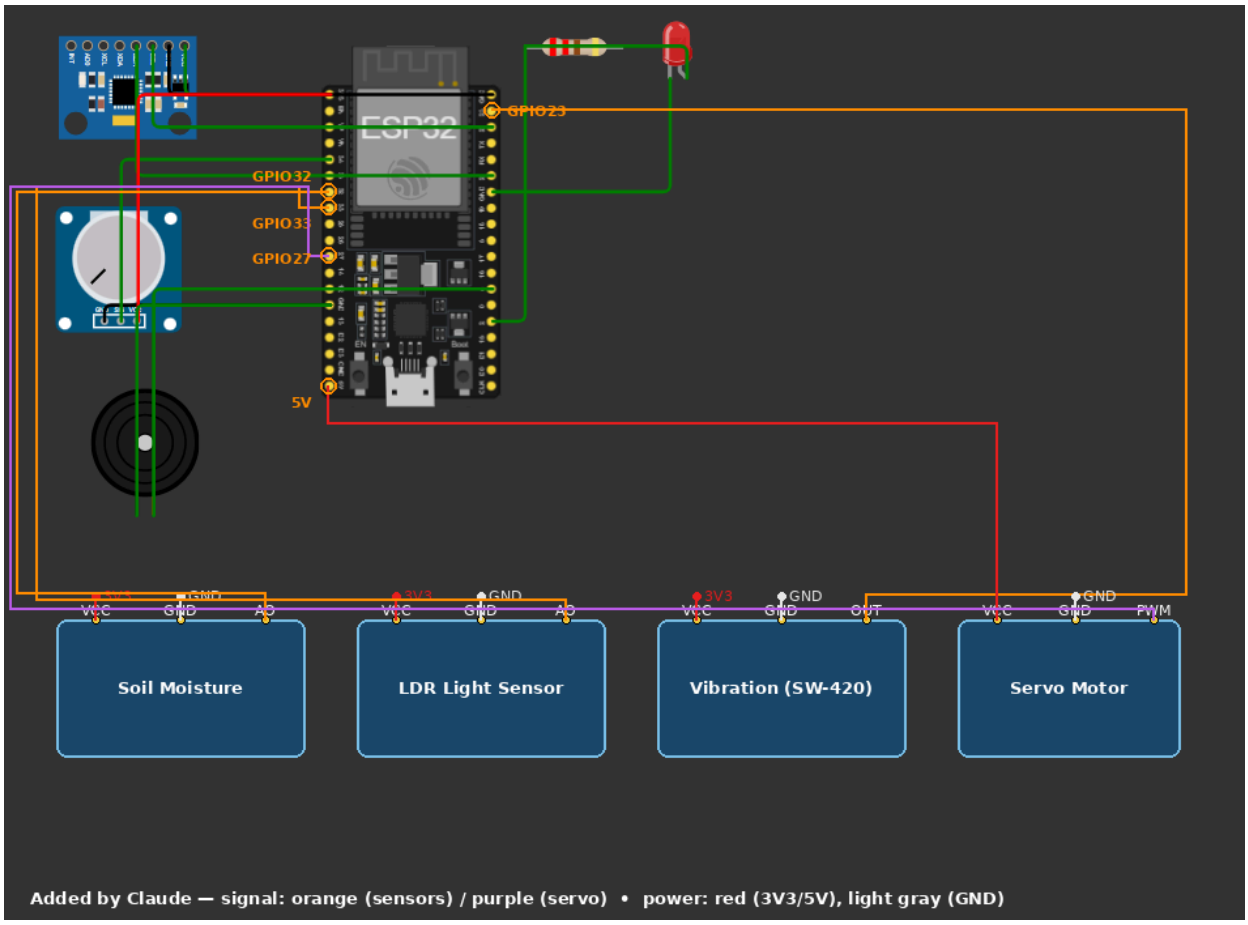
    }

    else {

      Serial.println(
        "Could not connect to CENTRAL."
      );
    }
  }


  Serial.println(
    "============================"
  );


  delay(2000);
}
```



CODE FOR NODE 2:

```
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <ESP32Servo.h>
#include <math.h>

Adafruit_MPU6050 mpu;
Servo warningServo;

const int NODE_ID = 2;

const char* ssid = "Landslide_Central";
const char* password = "12345678";
const char* centralIP = "192.168.4.1";

WiFiClient client;

// Pins
```

```
const int potPin = 34;
const int vibrationPin = 35;
const int ldrPin = 32;
const int soilPin = 33;

const int ledPin = 2;
const int buzzerPin = 4;
const int servoPin = 27;

// Thresholds
const int POT_THRESHOLD = 2500;
const int SOIL_THRESHOLD = 2500;
const float ACCELERATION_THRESHOLD = 12.0;

// Required signals
const int REQUIRED_SIGNALS = 2;

// Servo positions
const int SERVO_NORMAL = 0;
const int SERVO_WARNING = 90;


void setup() {

  Serial.begin(115200);

  pinMode(vibrationPin, INPUT);

  pinMode(ledPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);

  digitalWrite(ledPin, LOW);
  digitalWrite(buzzerPin, LOW);

  // Servo
  warningServo.setPeriodHertz(50);
  warningServo.attach(servoPin, 500, 2400);
  warningServo.write(SERVO_NORMAL);

  // I2C
  Wire.begin();

  // MPU6050
  if (!mpu.begin()) {
```

```
    Serial.println("MPU6050 NOT FOUND!");

    while (1) {
      delay(10);
    }
  }

  Serial.println("MPU6050 connected!");

  // Wi-Fi
  Serial.println();
  Serial.println("Connecting to Central Wi-Fi...");

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {

    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("Wi-Fi connected!");

  Serial.print("Node IP: ");
  Serial.println(WiFi.localIP());

  Serial.print("Central IP: ");
  Serial.println(centralIP);

  Serial.println();
  Serial.println("================================");
  Serial.println("       NODE 2 ONLINE");
  Serial.println("   MULTI-SENSOR DETECTION");
  Serial.println("================================");
}


void loop() {

  // Read sensors
  int potValue = analogRead(potPin);
  int vibrationValue = digitalRead(vibrationPin);
```

```
int lightValue = analogRead(ldrPin);
int soilValue = analogRead(soilPin);

// MPU6050
sensors_event_t a;
sensors_event_t g;
sensors_event_t temp;

mpu.getEvent(&a, &g, &temp);

// Total acceleration
float acceleration = sqrt(
  a.acceleration.x * a.acceleration.x +
  a.acceleration.y * a.acceleration.y +
  a.acceleration.z * a.acceleration.z
);

// Detection signals
bool groundMovement = false;
bool vibrationDetected = false;
bool wetSoil = false;
bool mpuMovement = false;

// Potentiometer
if (potValue > POT_THRESHOLD) {
  groundMovement = true;
}

// Vibration
if (vibrationValue == HIGH) {
  vibrationDetected = true;
}

// Soil
if (soilValue > SOIL_THRESHOLD) {
  wetSoil = true;
}

// MPU6050
if (acceleration > ACCELERATION_THRESHOLD) {
  mpuMovement = true;
}

// Count signals
```

```
int signalCount = 0;

if (groundMovement) {
  signalCount++;
}

if (vibrationDetected) {
  signalCount++;
}

if (wetSoil) {
  signalCount++;
}

if (mpuMovement) {
  signalCount++;
}

// Final decision
bool abnormal =
  signalCount >= REQUIRED_SIGNALS;

// Local alarm
if (abnormal) {

  digitalWrite(ledPin, HIGH);
  digitalWrite(buzzerPin, HIGH);

  warningServo.write(SERVO_WARNING);

} else {

  digitalWrite(ledPin, LOW);
  digitalWrite(buzzerPin, LOW);

  warningServo.write(SERVO_NORMAL);
}

// Serial monitor
Serial.println();
Serial.println("========== NODE 2 ==========");

Serial.print("Potentiometer : ");
Serial.println(potValue);
```

```
Serial.print("Vibration     : ");
Serial.println(vibrationValue);

Serial.print("Light         : ");
Serial.println(lightValue);

Serial.print("Soil moisture : ");
Serial.println(soilValue);

Serial.print("Acceleration  : ");
Serial.print(acceleration);
Serial.println(" m/s^2");

Serial.println("---------------------------");

Serial.print("Ground movement : ");
Serial.println(groundMovement ? "YES" : "NO");

Serial.print("Vibration       : ");
Serial.println(vibrationDetected ? "YES" : "NO");

Serial.print("Wet soil        : ");
Serial.println(wetSoil ? "YES" : "NO");

Serial.print("MPU movement    : ");
Serial.println(mpuMovement ? "YES" : "NO");

Serial.print("Detection signals = ");
Serial.println(signalCount);

Serial.print("NODE STATUS = ");

if (abnormal) {
  Serial.println("ABNORMAL");
} else {
  Serial.println("NORMAL");
}

// Send to Central
if (WiFi.status() == WL_CONNECTED) {

  if (client.connect(centralIP, 80)) {
```

```
      client.print("NODE=");
      client.print(NODE_ID);

      client.print(",POT=");
      client.print(potValue);

      client.print(",VIB=");
      client.print(vibrationValue);

      client.print(",LIGHT=");
      client.print(lightValue);

      client.print(",SOIL=");
      client.print(soilValue);

      client.print(",ACC=");
      client.print(acceleration);

      client.print(",SIGNALS=");
      client.print(signalCount);

      client.print(",ABNORMAL=");
      client.println(abnormal ? 1 : 0);

      client.stop();

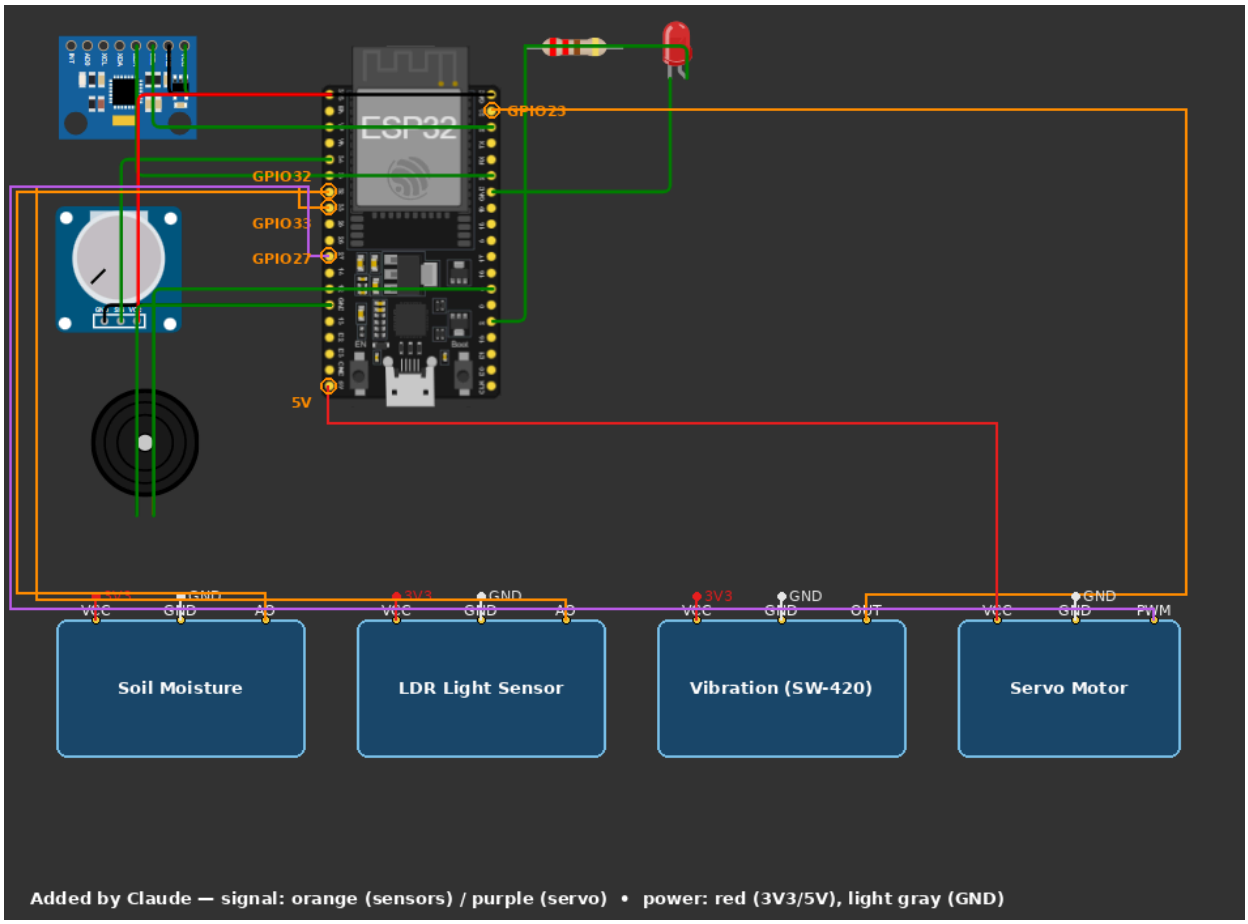
      Serial.println("Data sent to CENTRAL!");

    } else {

      Serial.println("Could not connect to CENTRAL.");
    }
  }

  Serial.println("============================");

  delay(2000);
}
```



CODE FOR NODE 3:

```
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <ESP32Servo.h>
#include <math.h>

Adafruit_MPU6050 mpu;
Servo warningServo;

const int NODE_ID = 3;

const char* ssid = "Landslide_Central";
const char* password = "12345678";
const char* centralIP = "192.168.4.1";

WiFiClient client;

// Pins
```

```
const int potPin = 34;
const int vibrationPin = 35;
const int ldrPin = 32;
const int soilPin = 33;

const int ledPin = 2;
const int buzzerPin = 4;
const int servoPin = 27;

// Thresholds
const int POT_THRESHOLD = 2500;
const int SOIL_THRESHOLD = 2500;
const float ACCELERATION_THRESHOLD = 12.0;

// Required signals
const int REQUIRED_SIGNALS = 2;

// Servo positions
const int SERVO_NORMAL = 0;
const int SERVO_WARNING = 90;


void setup() {

  Serial.begin(115200);

  pinMode(vibrationPin, INPUT);

  pinMode(ledPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);

  digitalWrite(ledPin, LOW);
  digitalWrite(buzzerPin, LOW);

  // Servo
  warningServo.setPeriodHertz(50);
  warningServo.attach(servoPin, 500, 2400);
  warningServo.write(SERVO_NORMAL);

  // I2C
  Wire.begin();

  // MPU6050
  if (!mpu.begin()) {
```

```
    Serial.println("MPU6050 NOT FOUND!");

    while (1) {
      delay(10);
    }
  }

  Serial.println("MPU6050 connected!");

  // Wi-Fi
  Serial.println();
  Serial.println("Connecting to Central Wi-Fi...");

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {

    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("Wi-Fi connected!");

  Serial.print("Node IP: ");
  Serial.println(WiFi.localIP());

  Serial.print("Central IP: ");
  Serial.println(centralIP);

  Serial.println();
  Serial.println("================================");
  Serial.println("       NODE 3 ONLINE");
  Serial.println("   MULTI-SENSOR DETECTION");
  Serial.println("================================");
}


void loop() {

  // Read sensors
  int potValue = analogRead(potPin);
  int vibrationValue = digitalRead(vibrationPin);
```

```
int lightValue = analogRead(ldrPin);
int soilValue = analogRead(soilPin);

// MPU6050
sensors_event_t a;
sensors_event_t g;
sensors_event_t temp;

mpu.getEvent(&a, &g, &temp);

// Total acceleration
float acceleration = sqrt(
  a.acceleration.x * a.acceleration.x +
  a.acceleration.y * a.acceleration.y +
  a.acceleration.z * a.acceleration.z
);

// Detection signals
bool groundMovement = false;
bool vibrationDetected = false;
bool wetSoil = false;
bool mpuMovement = false;

// Potentiometer
if (potValue > POT_THRESHOLD) {
  groundMovement = true;
}

// Vibration
if (vibrationValue == HIGH) {
  vibrationDetected = true;
}

// Soil
if (soilValue > SOIL_THRESHOLD) {
  wetSoil = true;
}

// MPU6050
if (acceleration > ACCELERATION_THRESHOLD) {
  mpuMovement = true;
}

// Count signals
```

```
int signalCount = 0;

if (groundMovement) {
  signalCount++;
}

if (vibrationDetected) {
  signalCount++;
}

if (wetSoil) {
  signalCount++;
}

if (mpuMovement) {
  signalCount++;
}

// Final decision
bool abnormal =
  signalCount >= REQUIRED_SIGNALS;

// Local alarm
if (abnormal) {

  digitalWrite(ledPin, HIGH);
  digitalWrite(buzzerPin, HIGH);

  warningServo.write(SERVO_WARNING);

} else {

  digitalWrite(ledPin, LOW);
  digitalWrite(buzzerPin, LOW);

  warningServo.write(SERVO_NORMAL);
}

// Serial monitor
Serial.println();
Serial.println("========== NODE 3 ==========");

Serial.print("Potentiometer : ");
Serial.println(potValue);
```

```
Serial.print("Vibration     : ");
Serial.println(vibrationValue);

Serial.print("Light         : ");
Serial.println(lightValue);

Serial.print("Soil moisture : ");
Serial.println(soilValue);

Serial.print("Acceleration  : ");
Serial.print(acceleration);
Serial.println(" m/s^2");

Serial.println("---------------------------");

Serial.print("Ground movement : ");
Serial.println(groundMovement ? "YES" : "NO");

Serial.print("Vibration       : ");
Serial.println(vibrationDetected ? "YES" : "NO");

Serial.print("Wet soil        : ");
Serial.println(wetSoil ? "YES" : "NO");

Serial.print("MPU movement    : ");
Serial.println(mpuMovement ? "YES" : "NO");

Serial.print("Detection signals = ");
Serial.println(signalCount);

Serial.print("NODE STATUS = ");

if (abnormal) {
  Serial.println("ABNORMAL");
} else {
  Serial.println("NORMAL");
}

// Send to Central
if (WiFi.status() == WL_CONNECTED) {

  if (client.connect(centralIP, 80)) {
```

```
      client.print("NODE=");
      client.print(NODE_ID);

      client.print(",POT=");
      client.print(potValue);

      client.print(",VIB=");
      client.print(vibrationValue);

      client.print(",LIGHT=");
      client.print(lightValue);

      client.print(",SOIL=");
      client.print(soilValue);

      client.print(",ACC=");
      client.print(acceleration);

      client.print(",SIGNALS=");
      client.print(signalCount);

      client.print(",ABNORMAL=");
      client.println(abnormal ? 1 : 0);

      client.stop();

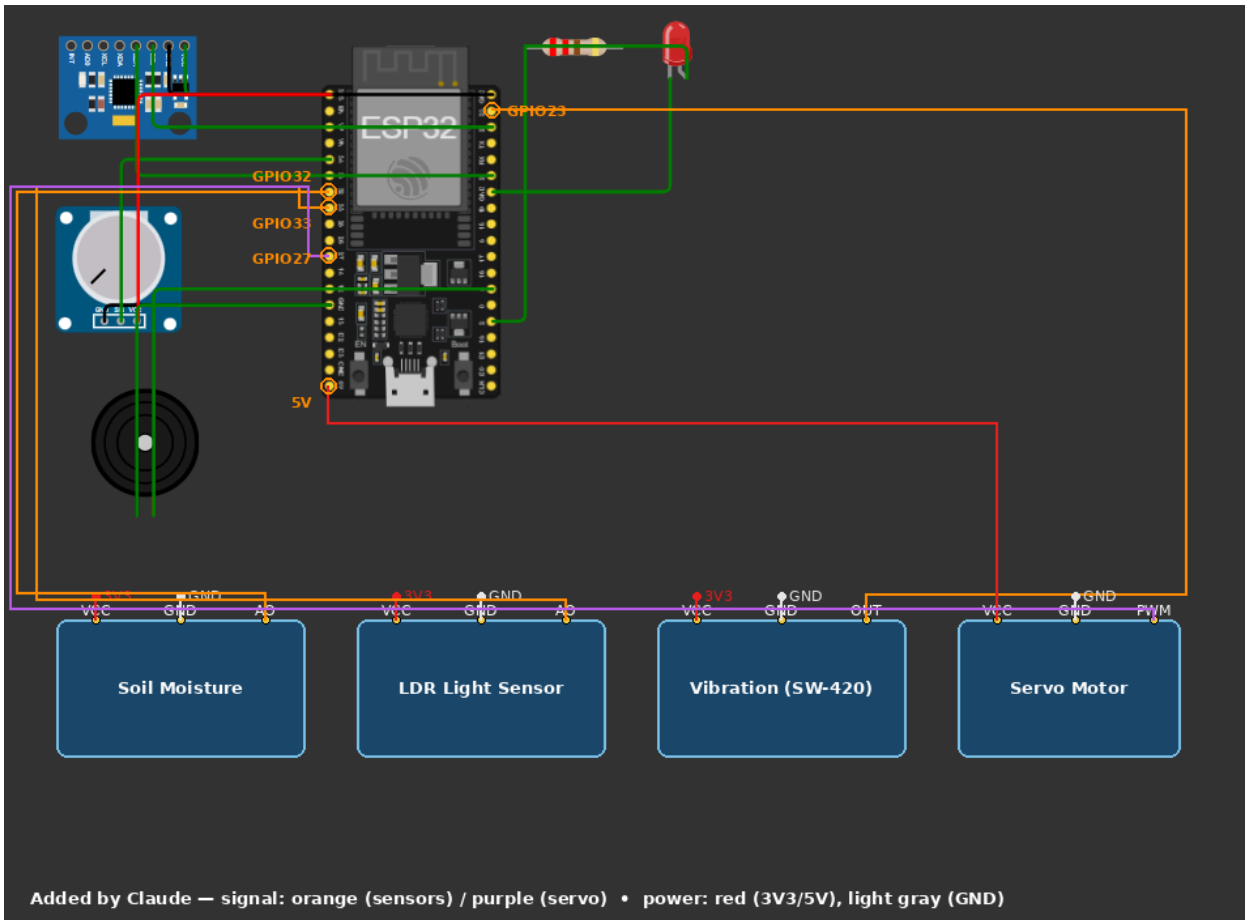
      Serial.println("Data sent to CENTRAL!");

    } else {

      Serial.println("Could not connect to CENTRAL.");
    }
  }

  Serial.println("============================");

  delay(2000);
}
```



CODE FOR NODE 4:

```
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <ESP32Servo.h>
#include <math.h>

Adafruit_MPU6050 mpu;
Servo warningServo;

const int NODE_ID = 4;

// ====================================================
// WIFI
// ====================================================

const char* ssid = "Landslide_Central";
const char* password = "12345678";
const char* centralIP = "192.168.4.1";
```

```
WiFiClient client;


// ======================================================
// PINS
// ======================================================

const int potPin = 34;
const int vibrationPin = 35;
const int ldrPin = 32;
const int soilPin = 33;

const int ledPin = 2;
const int buzzerPin = 4;
const int servoPin = 27;


// ======================================================
// THRESHOLDS
// ======================================================

const int POT_THRESHOLD = 2500;
const int SOIL_THRESHOLD = 2500;

const float ACCELERATION_THRESHOLD = 12.0;


// ======================================================
// REQUIRED NUMBER OF SIGNALS
// ======================================================

const int REQUIRED_SIGNALS = 2;


// ======================================================
// SERVO
// ======================================================

const int SERVO_NORMAL = 0;
const int SERVO_WARNING = 90;


// ======================================================
```

```
// SETUP
// ======================================================

void setup() {

  Serial.begin(115200);

  pinMode(vibrationPin, INPUT);

  pinMode(ledPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);

  digitalWrite(ledPin, LOW);
  digitalWrite(buzzerPin, LOW);


  // Servo
  warningServo.setPeriodHertz(50);

  warningServo.attach(
    servoPin,
    500,
    2400
  );

  warningServo.write(
    SERVO_NORMAL
  );


  // I2C
  Wire.begin();


  // MPU6050
  if (!mpu.begin()) {

    Serial.println(
      "MPU6050 NOT FOUND!"
    );

    while (1) {
      delay(10);
    }
```

```
}

Serial.println(
  "MPU6050 connected!"
);


// Wi-Fi
Serial.println();
Serial.println(
  "Connecting to Central Wi-Fi..."
);

WiFi.begin(
  ssid,
  password
);


while (
  WiFi.status() != WL_CONNECTED
) {

  delay(500);

  Serial.print(".");
}


Serial.println();

Serial.println(
  "Wi-Fi connected!"
);


Serial.print(
  "Node IP: "
);

Serial.println(
  WiFi.localIP()
);
```

```
  Serial.print(
    "Central IP: "
  );

  Serial.println(
    centralIP
  );


  Serial.println();

  Serial.println(
    "================================"
  );

  Serial.println(
    "       NODE 4 ONLINE"
  );

  Serial.println(
    "   MULTI-SENSOR DETECTION"
  );

  Serial.println(
    "================================"
  );
}


// ======================================================
// LOOP
// ======================================================

void loop() {

  // ==================================================
  // READ SENSORS
  // ==================================================

  int potValue =
    analogRead(potPin);

  int vibrationValue =
```

```
  digitalRead(vibrationPin);

int lightValue =
  analogRead(ldrPin);

int soilValue =
  analogRead(soilPin);


// ================================================
// MPU6050
// ================================================

sensors_event_t a;
sensors_event_t g;
sensors_event_t temp;

mpu.getEvent(
  &a,
  &g,
  &temp
);


// ================================================
// TOTAL ACCELERATION
// ================================================

float acceleration =
  sqrt(
    a.acceleration.x *
    a.acceleration.x +

    a.acceleration.y *
    a.acceleration.y +

    a.acceleration.z *
    a.acceleration.z
  );


// ================================================
// DETECTION SIGNALS
// ================================================
```

```
bool groundMovement = false;
bool vibrationDetected = false;
bool wetSoil = false;
bool mpuMovement = false;


// Potentiometer
if (
  potValue >
  POT_THRESHOLD
) {

  groundMovement = true;
}


// Vibration
if (
  vibrationValue == HIGH
) {

  vibrationDetected = true;
}


// Soil moisture
if (
  soilValue >
  SOIL_THRESHOLD
) {

  wetSoil = true;
}


// MPU6050
if (
  acceleration >
  ACCELERATION_THRESHOLD
) {

  mpuMovement = true;
}
```

```
// ================================================
// COUNT SIGNALS
// ================================================

int signalCount = 0;


if (groundMovement) {
  signalCount++;
}


if (vibrationDetected) {
  signalCount++;
}


if (wetSoil) {
  signalCount++;
}


if (mpuMovement) {
  signalCount++;
}


// ================================================
// FINAL NODE DECISION
// ================================================

bool abnormal =
  signalCount >= REQUIRED_SIGNALS;


// ================================================
// LOCAL WARNING
// ================================================

if (abnormal) {

  digitalWrite(
```

```
    ledPin,
    HIGH
  );

  digitalWrite(
    buzzerPin,
    HIGH
  );

  warningServo.write(
    SERVO_WARNING
  );

}

else {

  digitalWrite(
    ledPin,
    LOW
  );

  digitalWrite(
    buzzerPin,
    LOW
  );

  warningServo.write(
    SERVO_NORMAL
  );
}


// ================================================
// SERIAL MONITOR
// ================================================

Serial.println();

Serial.println(
  "========== NODE 4 =========="
);
```

```
Serial.print(
  "Potentiometer : "
);

Serial.println(
  potValue
);


Serial.print(
  "Vibration     : "
);

Serial.println(
  vibrationValue
);


Serial.print(
  "Light         : "
);

Serial.println(
  lightValue
);


Serial.print(
  "Soil moisture : "
);

Serial.println(
  soilValue
);


Serial.print(
  "Acceleration  : "
);

Serial.print(
  acceleration
);
```

```
Serial.println(
  " m/s^2"
);


Serial.println(
  "----------------------------"
);


Serial.print(
  "Ground movement : "
);

Serial.println(
  groundMovement
    ? "YES"
    : "NO"
);


Serial.print(
  "Vibration       : "
);

Serial.println(
  vibrationDetected
    ? "YES"
    : "NO"
);


Serial.print(
  "Wet soil        : "
);

Serial.println(
  wetSoil
    ? "YES"
    : "NO"
);


Serial.print(
```

```
  "MPU movement    : "
);

Serial.println(
  mpuMovement
    ? "YES"
    : "NO"
);


Serial.print(
  "Detection signals = "
);

Serial.println(
  signalCount
);


Serial.print(
  "NODE STATUS = "
);


if (abnormal) {

  Serial.println(
    "ABNORMAL"
  );

}

else {

  Serial.println(
    "NORMAL"
  );
}


// ================================================
// SEND TO CENTRAL
// ================================================
```

```
if (
  WiFi.status() ==
  WL_CONNECTED
) {

  if (
    client.connect(
      centralIP,
      80
    )
  ) {

    client.print(
      "NODE="
    );

    client.print(
      NODE_ID
    );


    client.print(
      ",POT="
    );

    client.print(
      potValue
    );


    client.print(
      ",VIB="
    );

    client.print(
      vibrationValue
    );


    client.print(
      ",LIGHT="
    );

    client.print(
```

```
  lightValue
);


client.print(
  ",SOIL="
);

client.print(
  soilValue
);


client.print(
  ",ACC="
);

client.print(
  acceleration
);


client.print(
  ",SIGNALS="
);

client.print(
  signalCount
);


client.print(
  ",ABNORMAL="
);

client.println(
  abnormal
    ? 1
    : 0
);


client.stop();
```

```
      Serial.println(
        "Data sent to CENTRAL!"
      );

    }

    else {

      Serial.println(
        "Could not connect to CENTRAL."
      );
    }
  }


  Serial.println(
    "============================"
  );


  delay(2000);
}
```

THE 4 NODES WILL BE CONNECTED TO A CENTRAL ESP32… IT WILL ACT AS THE WIFI ACCESS POINT.. THE 4 NODES WILL AUTOMATICALLY CONNECT TO IT:



CODE FOR CENTRAL ESP32:

```
#include <WiFi.h>

// ================================================
// WIFI
// ================================================

const char* ssid = "Landslide_Central";
const char* password = "12345678";

WiFiServer server(80);


// ================================================
// NODE STATUS
// ================================================

bool node1Abnormal = false;
bool node2Abnormal = false;
bool node3Abnormal = false;
bool node4Abnormal = false;


// ================================================
// LAST COMMUNICATION TIME
// ================================================
```

```
unsigned long node1LastSeen = 0;
unsigned long node2LastSeen = 0;
unsigned long node3LastSeen = 0;
unsigned long node4LastSeen = 0;


// ======================================================
// NODE TIMEOUT
// ======================================================

const unsigned long NODE_TIMEOUT = 10000;


// ======================================================
// SETUP
// ======================================================

void setup() {

  Serial.begin(115200);

  Serial.println();
  Serial.println("========================================");
  Serial.println("       LANDSLIDE CENTRAL");
  Serial.println("========================================");

  // Create Wi-Fi network
  WiFi.softAP(
    ssid,
    password
  );

  Serial.println("Wi-Fi network started!");

  Serial.print("Network: ");
  Serial.println(ssid);

  Serial.print("Central IP: ");
  Serial.println(
    WiFi.softAPIP()
  );

  // Start server
```

```
  server.begin();

  Serial.println("Server started!");

  Serial.println(
    "Waiting for Nodes 1-4..."
  );
}


// ======================================================
// MAIN LOOP
// ======================================================

void loop() {

  // Check for incoming node connection
  WiFiClient client = server.available();


  if (client) {

    unsigned long startTime = millis();


    // Wait for data
    while (
      !client.available() &&
      millis() - startTime < 2000
    ) {

      delay(1);
    }


    if (client.available()) {

      String data =
        client.readStringUntil('\n');

      data.trim();


      Serial.println();
```

```
Serial.println(
  "========== DATA RECEIVED =========="
);

Serial.println(data);


// ================================================
// GET NODE NUMBER
// ================================================

int nodeNumber =
  getValue(
    data,
    "NODE=",
    ','
  );


// ================================================
// GET ABNORMAL STATUS
// ================================================

int abnormal =
  getValue(
    data,
    "ABNORMAL=",
    '\n'
  );


// ================================================
// UPDATE NODE
// ================================================

if (nodeNumber == 1) {

  node1Abnormal =
    (abnormal == 1);

  node1LastSeen =
    millis();

  Serial.println(
```

```
    "Node 1 updated."
  );
}


else if (nodeNumber == 2) {

  node2Abnormal =
    (abnormal == 1);

  node2LastSeen =
    millis();

  Serial.println(
    "Node 2 updated."
  );
}


else if (nodeNumber == 3) {

  node3Abnormal =
    (abnormal == 1);

  node3LastSeen =
    millis();

  Serial.println(
    "Node 3 updated."
  );
}


else if (nodeNumber == 4) {

  node4Abnormal =
    (abnormal == 1);

  node4LastSeen =
    millis();

  Serial.println(
    "Node 4 updated."
  );
```

```
      }


      client.println(
        "DATA RECEIVED"
      );
    }


    client.stop();
  }


  // ========================================================
  // REMOVE OFFLINE NODES
  // ========================================================

  checkNodeTimeouts();


  // ========================================================
  // DISPLAY STATUS
  // ========================================================

  static unsigned long lastDisplay = 0;


  if (
    millis() - lastDisplay >= 3000
  ) {

    lastDisplay = millis();

    displaySystemStatus();
  }
}


// ========================================================
// GET VALUE FROM MESSAGE
// ========================================================

int getValue(
  String data,
```

```
  String key,
  char ending
) {

  int start =
    data.indexOf(key);


  if (start == -1) {
    return 0;
  }


  start += key.length();


  int end =
    data.indexOf(
      ending,
      start
    );


  if (end == -1) {

    end =
      data.length();
  }


  String value =
    data.substring(
      start,
      end
    );


  return value.toInt();
}


// ========================================================
// CHECK NODE TIMEOUTS
// ========================================================
```

```
void checkNodeTimeouts() {

  unsigned long now =
    millis();


  if (
    node1LastSeen > 0 &&
    now - node1LastSeen > NODE_TIMEOUT
  ) {

    node1Abnormal = false;
  }


  if (
    node2LastSeen > 0 &&
    now - node2LastSeen > NODE_TIMEOUT
  ) {

    node2Abnormal = false;
  }


  if (
    node3LastSeen > 0 &&
    now - node3LastSeen > NODE_TIMEOUT
  ) {

    node3Abnormal = false;
  }


  if (
    node4LastSeen > 0 &&
    now - node4LastSeen > NODE_TIMEOUT
  ) {

    node4Abnormal = false;
  }
}
```

```
// ================================================
// DISPLAY SYSTEM STATUS
// ================================================

void displaySystemStatus() {

  int abnormalNodes = 0;


  // Count abnormal nodes
  if (node1Abnormal) {
    abnormalNodes++;
  }

  if (node2Abnormal) {
    abnormalNodes++;
  }

  if (node3Abnormal) {
    abnormalNodes++;
  }

  if (node4Abnormal) {
    abnormalNodes++;
  }


  // ================================================
  // DISPLAY NODE INFORMATION
  // ================================================

  Serial.println();
  Serial.println();
  Serial.println(
    "======================================"
  );

  Serial.println(
    "        LANDSLIDE MONITOR"
  );

  Serial.println(
    "======================================"
  );
```

```
Serial.print("Node 1 : ");

Serial.println(
  node1Abnormal
    ? "ABNORMAL"
    : "NORMAL"
);


Serial.print("Node 2 : ");

Serial.println(
  node2Abnormal
    ? "ABNORMAL"
    : "NORMAL"
);


Serial.print("Node 3 : ");

Serial.println(
  node3Abnormal
    ? "ABNORMAL"
    : "NORMAL"
);


Serial.print("Node 4 : ");

Serial.println(
  node4Abnormal
    ? "ABNORMAL"
    : "NORMAL"
);


Serial.println(
  "-----------------------------------"
);


Serial.print(
```

```
  "ABNORMAL NODES = "
);

Serial.println(
  abnormalNodes
);


Serial.println(
  "-------------------------------------"
);


// ================================================
// FINAL DECISION
// ================================================

if (abnormalNodes == 0) {

  Serial.println(
    "SYSTEM STATUS: NORMAL"
  );

  Serial.println(
    "No abnormal nodes detected."
  );
}


else if (abnormalNodes == 1) {

  Serial.println(
    "SYSTEM STATUS: LOW WARNING"
  );

  Serial.println(
    "One node detected abnormality."
  );
}


else if (abnormalNodes == 2) {

  Serial.println(
```

```
    "SYSTEM STATUS: WARNING"
  );

  Serial.println(
    "Two nodes detected abnormality."
  );
}


else if (abnormalNodes == 3) {

  Serial.println(
    "SYSTEM STATUS: HIGH RISK"
  );

  Serial.println(
    "Three nodes detected abnormality!"
  );

  Serial.println(
    "LANDSLIDE CONDITIONS ARE SERIOUS."
  );
}


else {

  Serial.println(
    "SYSTEM STATUS: CRITICAL"
  );

  Serial.println(
    "ALL FOUR NODES ARE ABNORMAL!"
  );

  Serial.println(
    "CRITICAL LANDSLIDE WARNING!"
  );
}


Serial.println(
  "======================================"
);
```

}

!!!Central receives data from all 4 nodes , remember each node's latest status , count abnormal nodes,display the overall situation
!!!

COMPLETE OVERVIEW OF WHAT THE HARDWARE WILL DO:
We have **4 identical sensor nodes** placed at different points on the slope.
Each node has its own **ESP32**.
Each node contains:

- **MPU6050** → detects movement/acceleration
- **Vibration sensor** → detects vibration
- **Potentiometer** → simulates ground displacement
- **Soil-moisture sensor** → measures soil moisture
- **Light sensor/LDR** → measures light
- **LED** → that node's warning indicator
- **Buzzer** → that node's warning sound
- **Servo motor** → that node's physical warning indicator

Each ESP32 continuously reads its sensors.
The node checks for abnormal conditions using:

- Ground movement
- Vibration
- Soil moisture
- MPU6050 movement

2 or more abnormal sensor signals within a node → that node becomes ABNORMAL.
When a node becomes ABNORMAL:

- Its **own LED turns ON**
- Its **own buzzer turns ON**
- Its **own servo moves**
- It sends its abnormal status to the Central ESP32.

The Central ESP32 receives the status from **all 4 nodes**.
The Central counts how many nodes are ABNORMAL.

1 abnormal node → not considered serious.

- No special system-wide alarm.
- Other nodes continue monitoring.

2 abnormal nodes → still not considered serious.

- No special system-wide alarm.
- The system continues monitoring.

3 abnormal nodes → HIGH RISK.

- The 3 affected nodes have their **own LED + buzzer + servo active**.
- Central reports HIGH RISK.
- SMS triggers

4 abnormal nodes → CRITICAL.

- **All 4 nodes' own LEDs + buzzers + servos** are active.
- Central reports CRITICAL.
- SMS is triggered.

There is **NO separate Central buzzer or Central LED** in current hardware design.